ASD Lab Notes

Lab 2

Summary

What This Lab Is Really About

Think of this like building a **universal remote control** for different types of encryption. Instead of having separate remotes for your TV, sound system, and lights, you want one remote that can control everything.

Key Learning Outcomes

1. Composition Over Inheritance

Simple analogy: Instead of building a new car from scratch every time, you take existing parts (engine, wheels, steering wheel) and combine them to make different types of cars.

In your code:

- RSACypher uses existing Java parts (Cipher, KeyPair)
- It doesn't reinvent encryption - it just combines existing pieces

2. The Factory Pattern

Think of it like a vending machine:

- You press "A1" and get chips
- You press "B2" and get soda
- The machine handles all the complex stuff inside

Your CypherFactory:

- You ask for CypherType.AES and get an AES encryptor
- You ask for CypherType.RSA and get an RSA encryptor
- The factory handles creating the right type

3. Abstraction and Interfaces

Like a universal plug adapter:

- Different countries have different plugs (AES, DES, RSA)
- But they all need to do the same thing (encrypt/decrypt)
- The Cypherable interface is like the universal standard

What Each File Does (In Plain English)

1. **Cypherable interface**: The "contract" - says "any encryptor must be able to encrypt and decrypt"
2. **AbstractCypher**: The "common stuff" - handles the basic encrypt/decrypt operations that all encryptors share
3. **SymmetricCypher**: For algorithms where the same key locks AND unlocks (like a house key)
4. **RSACypher**: For algorithms where you have two keys - one to lock, one to unlock (like a mailbox)
5. **CypherFactory**: The "smart creator" - you tell it what type you want, it builds it for you
6. **TestRunner**: Your "test drive" - shows everything working together

The Big Picture Learning Goals

1. **Code Reuse**: Don't write the same code twice - build once, use everywhere
2. **Flexibility**: Easy to add new encryption types without breaking existing code
3. **Separation of Concerns**: Each class has ONE job and does it well
4. **Loose Coupling**: Classes work together but don't depend too heavily on each other's internals